

4.4 Composite pattern

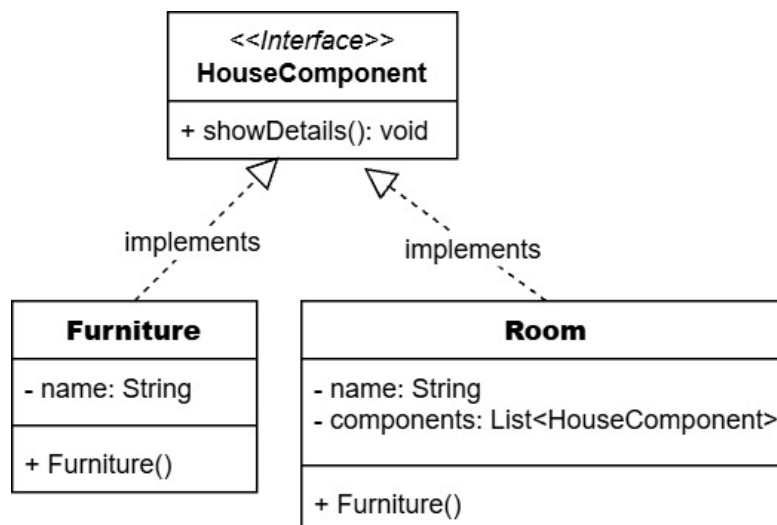
Composite Design Pattern ใช้จัดการโครงสร้างแบบลำดับชั้น (Hierarchy) โดยการใช้วัตถุแบบกลุ่ม (Composite) และวัตถุแบบเดี่ยว (Leaf) ที่มีโครงสร้างเหมือนกัน ช่วยให้สามารถจัดการทั้งกลุ่มของวัตถุ และวัตถุเดี่ยวได้ด้วยอินเทอร์เฟซเดียวกัน คล้ายการต่อเลโก้ คือประกอบชิ้นส่วนที่มีอยู่ให้เป็นรูปร่างต่าง ๆ

4.4.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Composite Design Pattern

- การจัดการโครงสร้างแบบลำดับชั้น (Hierarchy) เช่น โครงสร้างบ้านที่มีส่วนประกอบเป็นห้องต่าง ๆ แต่ละห้องอาจมีเฟอร์นิเจอร์หรือส่วนประกอบย่อยอื่น ๆ
- การทำงานกับวัตถุที่มีทั้งแบบเดี่ยวและแบบกลุ่ม เช่น การประมวลผลข้อมูลในระบบที่ต้องรองรับทั้งวัตถุเดี่ยวและกลุ่มของวัตถุ
- ลดความซับซ้อนของโค้ด ช่วยให้โค้ดหลักไม่ต้องตรวจสอบว่าวัตถุเป็นประเภทใด (เดี่ยวหรือกลุ่ม)
- เพิ่มความยืดหยุ่นในการขยาย สามารถเพิ่มประเภทของวัตถุ หรือขยายโครงสร้างได้ง่ายโดยไม่กระทบกับโค้ดเดิม

ตัวอย่าง แอปพลิเคชันที่จัดการโครงสร้างของบ้าน ซึ่งบ้านมีห้องต่าง ๆ เช่น ห้องนอน ห้องครัว และเฟอร์นิเจอร์ ในแต่ละห้อง เราต้องการโค้ดที่สามารถจัดการทั้งห้องเดี่ยว และกลุ่มของห้องได้อย่างยืดหยุ่น



ภาพที่ xx Composite design pattern

Interface HouseComponent กำหนดอินเทอร์เฟซสำหรับองค์ประกอบทุกประเภทในบ้าน เช่น ห้อง หรือเฟอร์นิเจอร์ มีเมธอด showDetails() สำหรับแสดงรายละเอียดขององค์ประกอบ

```
1. // Step 1: สร้าง Component Interface
2. interface HouseComponent {
3.     void showDetails();
4. }
```

Leaf Class (Furniture) ใช้แทนองค์ประกอบเดี่ยว เช่น เฟอร์นิเจอร์ Implement เมธอด showDetails() เพื่อแสดงชื่อเฟอร์นิเจอร์

```
1. // Step 2: สร้าง Leaf Class สำหรับองค์ประกอบเดี่ยว เช่น เฟอร์นิเจอร์
2. class Furniture implements HouseComponent {
3.
4.     private String name;
5.
6.     public Furniture(String name) {
7.         this.name = name;
8.     }
9.
10.    @Override
11.    public void showDetails() {
12.        System.out.println("Furniture: " + name);
13.    }
14. }
```

Composite Class (Room) ใช้แทนองค์ประกอบแบบกลุ่ม เช่น ห้อง มีเมธอด addComponent() และ removeComponent() สำหรับเพิ่มหรือเอาองค์ประกอบออกจากกลุ่ม Implement เมธอด showDetails() เพื่อแสดงข้อมูลของห้องและองค์ประกอบย่อย

```
1. // Step 3: สร้าง Composite Class สำหรับองค์ประกอบกลุ่ม (เช่น ห้อง)
2. class Room implements HouseComponent {
3.
4.     private String name;
5.     private List<HouseComponent> components = new ArrayList<>();
6.
7.     public Room(String name) {
8.         this.name = name;
9.     }
10.
11.    public void addComponent(HouseComponent component) {
12.        components.add(component);
13.    }
14.
15.    public void removeComponent(HouseComponent component) {
16.        components.remove(component);
17.    }
18.
19.    @Override
20.    public void showDetails() {
```

```

21.         System.out.println("Room: " + name);
22.         for (HouseComponent component : components) {
23.             component.showDetails();
24.         }
25.     }
26. }

```

Main Class สร้างโครงสร้างบ้านที่มีห้องและเฟอร์นิเจอร์ เรียกใช้เมธอด showDetails() เพื่อแสดงโครงสร้างทั้งหมด

```

1. // Step 4: การใช้งาน Composite Pattern
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้างเฟอร์นิเจอร์
6.         Furniture bed = new Furniture("Bed");
7.         Furniture wardrobe = new Furniture("Wardrobe");
8.         Furniture stove = new Furniture("Stove");
9.         Furniture fridge = new Furniture("Fridge");
10.
11.        // สร้างห้อง
12.        Room bedroom = new Room("Bedroom");
13.        bedroom.addComponent(bed);
14.        bedroom.addComponent(wardrobe);
15.
16.        Room kitchen = new Room("Kitchen");
17.        kitchen.addComponent(stove);
18.        kitchen.addComponent(fridge);
19.
20.        // สร้างบ้าน
21.        Room house = new Room("House");
22.        house.addComponent(bedroom);
23.        house.addComponent(kitchen);
24.
25.        // แสดงโครงสร้างบ้าน
26.        house.showDetails();
27.    }
28. }

```

ผลลัพธ์

Room: House

Room: Bedroom

Furniture: Bed

Furniture: Wardrobe

Room: Kitchen

Furniture: Stove

Furniture: Fridge

สถานการณ์ที่เหมาะสมกับ Composite Design Pattern สำหรับบ้าน

- โครงสร้างบ้านที่มีลำดับชั้น เช่น บ้าน -> ห้อง -> เฟอร์นิเจอร์

- ระบบที่ต้องจัดการทั้งอีอบเจกต์เดี่ยว และอีอบเจกต์แบบกลุ่ม เช่น การเพิ่มหรือเอาเฟอร์นิเจอร์ออกจากห้อง และการเพิ่มห้องเข้าในบ้าน
- การสร้างระบบจัดการข้อมูลแบบลำดับชั้น เช่น การออกแบบโครงสร้างเมนู หรือแผนผังองค์กร

ข้อดีของ Composite Design Pattern

- ลดความซับซ้อนของโค้ด ช่วยให้ไม่ต้องแยกโค้ดสำหรับการจัดการอีอบเจกต์เดี่ยว และอีอบเจกต์แบบกลุ่ม
- เพิ่มความยืดหยุ่นในการขยายระบบ สามารถเพิ่มประเภทขององค์ประกอบได้ง่าย โดยไม่กระทบกับโครงสร้างเดิม
- รองรับการทำงานแบบ Recursive เช่น การแสดงรายละเอียดของบ้านที่มีหลายชั้น สามารถใช้เมธอดเดียวกันสำหรับองค์ประกอบทั้งหมด

4.4.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Composite Pattern เหมาะสำหรับปัญหาที่ต้องจัดการโครงสร้างแบบลำดับชั้น (Hierarchy) ตัวอย่างปัญหาในงานจริงที่สามารถช่วยแก้ไขได้ มีดังนี้

- 1) การจัดการแผนผังองค์กร (Organization Hierarchy) องค์กรประกอบด้วยหลายระดับ เช่น ผู้บริหาร หัวหน้าแผนก และพนักงาน ซึ่งสามารถมีโครงสร้างแบบลำดับชั้น การเขียนโค้ดเพื่อจัดการแต่ละระดับแยกกันอาจซับซ้อนและยากต่อการบำรุงรักษา

ใช้ Composite Pattern เพื่อจัดการแผนผังองค์กร โดยสามารถแสดงข้อมูลของทั้งองค์กร แผนก หรือพนักงานแต่ละคนด้วยวิธีเดียวกัน

```

1. // Component Interface
2. interface Employee {
3.     void showDetails();
4. }

1. // Leaf Class
2. class IndividualEmployee implements Employee {
3.
4.     private String name;
5.     private String position;
6.
7.     public IndividualEmployee(String name, String position) {
8.         this.name = name;
9.         this.position = position;
10.    }
11.
12.    @Override
13.    public void showDetails() {
14.        System.out.println(position + ": " + name);
15.    }
16. }
```

```

1. // Composite Class
2. class Department implements Employee {
3.
4.     private String name;
5.     private List<Employee> employees = new ArrayList<>();
6.
7.     public Department(String name) {
8.         this.name = name;
9.     }
10.
11.     public void addEmployee(Employee employee) {
12.         employees.add(employee);
13.     }
14.
15.     public void removeEmployee(Employee employee) {
16.         employees.remove(employee);
17.     }
18.
19.     @Override
20.     public void showDetails() {
21.         System.out.println("Department: " + name);
22.         for (Employee employee : employees) {
23.             employee.showDetails();
24.         }
25.     }
26. }

1. // Main Class
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้าง Employees
6.         Employee emp1 = new IndividualEmployee("Alice", "Manager");
7.         Employee emp2 = new IndividualEmployee("Bob", "Developer");
8.         Employee emp3 = new IndividualEmployee("Charlie", "Tester");
9.
10.        // สร้าง Department
11.        Department engineering = new Department("Engineering");
12.        engineering.addEmployee(emp1);
13.        engineering.addEmployee(emp2);
14.        engineering.addEmployee(emp3);
15.
16.        // แสดงรายละเอียด Organization
17.        engineering.showDetails();
18.    }
19. }

```

- 2) การออกแบบเมนูในระบบ (Menu System) ระบบต้องจัดการเมนูหลายระดับ เช่น เมนูหลัก และเมนูย่อย การเขียนโค้ดเพื่อจัดการเมนูแต่ละระดับแยกกันทำให้เพิ่มความซับซ้อน และยากต่อการขยาย สามารถใช้ Composite Pattern เพื่อจัดการเมนูและเมนูย่อยในโครงสร้างแบบลำดับชั้น

4.5 Facade pattern

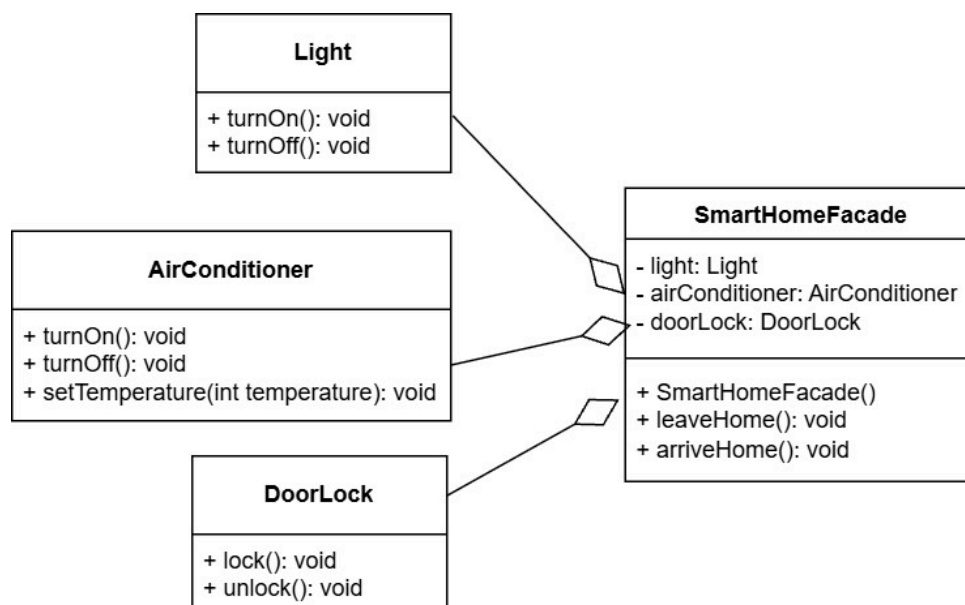
Facade Design Pattern เป็นหนึ่งในแพตเทิร์นโครงสร้างที่มีจุดประสงค์เพื่อซ่อนความซับซ้อนของระบบ โดยการสร้างอินเทอร์เฟซสำหรับการใช้งานระบบ ช่วยให้นักพัฒนาใช้งานระบบได้ง่ายขึ้น โดยไม่ต้องเข้าใจรายละเอียดภายในของระบบ คล้ายกับ Facade ด้านหน้าอาคาร คือมีเพื่อปิดบังความยุ่งเหยิงที่อยู่ด้านใน

4.5.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Facade Design Pattern

- ลดความซับซ้อนในการใช้งานระบบ ระบบที่มีโมดูลย่อยหรือส่วนประกอบจำนวนมาก มีความซับซ้อนในการใช้งาน Facade ช่วยรวบรวมและจัดระเบียบ ทำให้การเรียกใช้งานผ่านอินเทอร์เฟซเดียว
- ปรับปรุงการบำรุงรักษา การเปลี่ยนแปลงภายในระบบจะไม่กระทบกับผู้ใช้งาน เนื่องจากผู้ใช้งานเรียกใช้งานผ่าน Facade เท่านั้น
- ปรับปรุงการอ่านและเข้าใจโค้ด ลดจำนวนการเรียกใช้งานที่กระจัดกระจายไปยังโมดูลย่อยต่าง ๆ ทำให้โค้ดเรียบง่าย และเข้าใจง่าย
- ส่งเสริม Encapsulation ช่วยซ่อนรายละเอียดภายในระบบ ทำให้ระบบมีการแยกส่วนที่ดี และลดการพึ่งพาระหว่างโมดูล

ตัวอย่าง ระบบบ้านอัจฉริยะมีอุปกรณ์หลายชนิด เช่น ไฟ เครื่องปรับอากาศ และระบบล็อกประตู การควบคุมอุปกรณ์เหล่านี้ อาจซับซ้อนและยุ่งยากสำหรับผู้ใช้งาน Facade Pattern ช่วยสร้างอินเทอร์เฟซที่ง่ายเพื่อให้ผู้ใช้ควบคุมบ้านอัจฉริยะได้ด้วยคำสั่งเดียว



ภาพที่ xx Facade design pattern

Subsystems (Light, AirConditioner, DoorLock) เป็นคลาสที่แทนโมดูลย่อยในระบบบ้านอัจฉริยะ เช่น ไฟ เครื่องปรับอากาศ และระบบล็อกประตู

```
1. // Step 1: สร้าง Subsystems (โมดูลย่อย)
2. class Light {
3.
4.     public void turnOn() {
5.         System.out.println("Lights are ON");
6.     }
7.
8.     public void turnOff() {
9.         System.out.println("Lights are OFF");
10.    }
11. }

1. class AirConditioner {
2.
3.     public void turnOn() {
4.         System.out.println("Air Conditioner is ON");
5.     }
6.
7.     public void turnOff() {
8.         System.out.println("Air Conditioner is OFF");
9.     }
10.
11.     public void setTemperature(int temperature) {
12.         System.out.println("Air Conditioner temperature set to "
13.             + temperature + "°C");
14.     }
15. }

1. class DoorLock {
2.
3.     public void lock() {
4.         System.out.println("Doors are LOCKED");
5.     }
6.
7.     public void unlock() {
8.         System.out.println("Doors are UNLOCKED");
9.     }
10. }
```

SmartHomeFacade เป็นคลาส Facade ที่รวบรวมและจัดการการเรียกใช้งานโมดูลย่อยทั้งหมด มีเมธอด leaveHome() และ arriveHome() ที่รวมคำสั่งหลายคำสั่งให้เป็นคำสั่งเดียวสำหรับผู้ใช้

```
1. // Step 2: สร้าง Facade
2. class SmartHomeFacade {
3.
4.     private Light light;
5.     private AirConditioner airConditioner;
6.     private DoorLock doorLock;
```

```

7.
8.     public SmartHomeFacade() {
9.         this.light = new Light();
10.        this.airConditioner = new AirConditioner();
11.        this.doorLock = new DoorLock();
12.    }
13.
14.    public void leaveHome() {
15.        System.out.println("Leaving home...");
16.        light.turnOff();
17.        airConditioner.turnOff();
18.        doorLock.lock();
19.    }
20.
21.    public void arriveHome() {
22.        System.out.println("Arriving home...");
23.        doorLock.unlock();
24.        light.turnOn();
25.        airConditioner.turnOn();
26.        airConditioner.setTemperature(22);
27.    }
28. }

```

Main ผู้ใช้สามารถควบคุมระบบบ้านอัจฉริยะได้ง่าย โดยเรียกใช้งานผ่าน SmartHomeFacade เท่านั้น

```

1. // Step 3: การใช้งาน Facade
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้าง Smart Home Facade
6.         SmartHomeFacade smartHome = new SmartHomeFacade();
7.
8.         // ผู้ใช้ควบคุมบ้านได้ง่าย
9.         smartHome.arriveHome();
10.        System.out.println();
11.        smartHome.leaveHome();
12.    }
13. }

```

ผลลัพธ์

Arriving home...

Doors are UNLOCKED

Lights are ON

Air Conditioner is ON

Air Conditioner temperature set to 22°C

Leaving home...

Lights are OFF

Air Conditioner is OFF

Doors are LOCKED

4.5.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Facade Pattern เหมาะสำหรับการแก้ปัญหาในงานจริงที่ต้องการลดความซับซ้อนในการใช้งานระบบที่มีหลายโมดูลย่อย ตัวอย่างปัญหาที่สามารถแก้ไขได้ด้วย Facade Pattern มีดังนี้

1) ระบบจัดการเอกสาร (Document Management System)

ระบบจัดการเอกสารต้องรองรับการจัดการไฟล์ที่ต้องทำหลายงานเพื่อให้จัดการได้สำเร็จ เช่น อ่านเอกสาร บีบอัดไฟล์ และส่งไฟล์ผ่านอีเมล ซึ่งแต่ละฟังก์ชันต้องเรียกใช้ API ของโมดูลย่อยที่แตกต่างกัน ทำให้โค้ดซับซ้อนและยากต่อการบำรุงรักษา

ใช้ Facade Pattern เพื่อสร้าง DocumentManagerFacade ที่รวมการเรียกใช้ฟังก์ชันต่าง ๆ และซ่อนรายละเอียดการทำงานภายใน

```
1. // Subsystems
2. class DocumentReader {
3.
4.     public void read(String fileName) {
5.         System.out.println("Reading document: " + fileName);
6.     }
7. }

1. class FileCompressor {
2.     public void compress(String fileName) {
3.         System.out.println("Compressing file: " + fileName);
4.     }
5. }

1. class EmailSender {
2.     public void send(String fileName, String recipient) {
3.         System.out.println("Sending file: " + fileName + " to "
4.                             + recipient);
5.     }
6. }

1. // Facade
2. class DocumentManagerFacade {
3.
4.     private DocumentReader reader;
5.     private FileCompressor compressor;
6.     private EmailSender sender;
7.
8.     public DocumentManagerFacade() {
9.         this.reader = new DocumentReader();
```

```

10.         this.compressor = new FileCompressor();
11.         this.sender = new EmailSender();
12.     }
13.
14.     public void processAndSend(String fileName, String recipient) {
15.         reader.read(fileName);
16.         compressor.compress(fileName);
17.         sender.send(fileName, recipient);
18.     }
19. }

1. // Main
2. public class Main {
3.     public static void main(String[] args) {
4.         DocumentManagerFacade manager = new DocumentManagerFacade();
5.         manager.processAndSend("example.doc", "user@example.com");
6.     }
7. }

```

ผลลัพธ์

Reading document: example.doc

Compressing file: example.doc

Sending file: example.doc to user@example.com

2) ระบบซื้อขายออนไลน์ (E-Commerce System)

ระบบ E-Commerce มีโมดูลย่อยหลายตัว เช่น ระบบตรวจสอบสินค้า ระบบชำระเงิน และ ระบบจัดส่งสินค้า การจัดการคำสั่งซื้อแต่ละครั้งต้องเรียกใช้โมดูลเหล่านี้ที่ละตัว ซึ่งทำให้โค้ดซับซ้อน

ใช้ Facade Pattern เพื่อสร้าง OrderProcessingFacade ที่รวมการจัดการคำสั่งซื้อไว้ในที่เดียว

```

1. // Subsystems
2. class Inventory {
3.
4.     public boolean checkStock(String product) {
5.         System.out.println("Checking stock for product: " + product);
6.         return true;
7.     }
8. }

1. class PaymentProcessor {
2.     public void processPayment(String paymentDetails) {
3.         System.out.println("Processing payment: " + paymentDetails);
4.     }
5. }

1. class Shipping {
2.     public void arrangeShipping(String product) {

```

```

3.         System.out.println("Arranging shipping for product: "
4.                               + product);
5.     }
6. }

1. // Facade
2. class OrderProcessingFacade {
3.
4.     private Inventory inventory;
5.     private PaymentProcessor paymentProcessor;
6.     private Shipping shipping;
7.
8.     public OrderProcessingFacade() {
9.         this.inventory = new Inventory();
10.        this.paymentProcessor = new PaymentProcessor();
11.        this.shipping = new Shipping();
12.    }
13.
14.    public void processOrder(String product, String paymentDetails)
15.    {
16.        if (inventory.checkStock(product)) {
17.            paymentProcessor.processPayment(paymentDetails);
18.            shipping.arrangeShipping(product);
19.            System.out.println("Order processed successfully!");
20.        } else {
21.            System.out.println("Product is out of stock!");
22.        }
23.    }
24. }

1. // Main
2. public class Main {
3.     public static void main(String[] args) {
4.         OrderProcessingFacade orderFacade = new
5.             OrderProcessingFacade();
6.         orderFacade.processOrder("Laptop", "Credit Card");
7.     }
8. }

```

ผลลัพธ์

Checking stock for product: Laptop
 Processing payment: Credit Card
 Arranging shipping for product: Laptop
 Order processed successfully!